Applied Intelligence (2023) 53:8673–8692
https://doi.org/10.1007/s10489-023-04523-6

Continuous build outcome prediction: an experimental evaluation and acceptance modelling

Marcin Kawalerowicz[1,2] · **Lech Madeyski**[3]

Accepted: 9 February 2023 / Published online: 18 April 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Continuous Build Outcome Prediction (CBOP) is a lightweight implementation of Continuous Defect Prediction (CDP). CBOP combines: 1) results of continuous integration (CI) and 2) the data mined from the version control system with 3) machine learning (ML) to form a practice that evolved from software defect prediction (SDP) where a failing build is treated as a defect to fight against. Here, we explain the CBOP idea, where we use historical build results together with metrics derived from a software repository to create a model that classifies changes the developer is introducing to the source code during her work in a just-in-time manner. To evaluate the CBOP idea, we perform a small-n repeated measure with two conditions and replicate experiment in a real-life, business-driven software project. In this preliminary evaluation of CBOP, we study whether the practice will reduce the Failed Build Ratio (FBR) - the ratio of failing build results to all other build results. We calculate effect size and p-value of change in FBR while using the CBOP practice, provide an analysis of our model, and perform and report the results of a Technology Acceptance Model (TAM)-inspired survey that we conducted among experiment participants and industry specialists to assess the acceptance of CBOP and the tool.

Keywords Software defect prediction · Agile experimentation · Continuous integration · Machine learning · Technology acceptance model

1 Introduction

While increasing complexity of software systems poses challenges that need to be addressed by software engineering processes, techniques, and tools, the proposed approaches need to be evaluated against earlier ones. In prior work [41] we proposed Continuous Defect Prediction (CDP), a new technique that has its roots in software defect prediction (SDP) and aids software developers in producing better quality code. This paper is an extension of our paper [31] where we proposed a lightweight implementation of CDP termed Continuous Build Outcome Prediction (CBOP). In CBOP, we classify build outcome based on historical Continuous Integration (CI) data about the build success or failure and metrics derived from the source code. CBOP builds on the SDP which is used to identify error-prone source code parts like files, classes, or even methods. In CBOP, a software change (e.g., push to the repository) is used to identify a possible defect that manifests itself in a broken build.

In a previous study we implemented CBOP in a software suite called Jaskier [32]. This software was used to perform a preliminary empirical evaluation of the CBOP practice in real settings described in [31].

Here, we advance this study by:

1. extending the experiment plan description; explaining the procedure we used when choosing the experiment design type,

This article belongs to the Topical Collection: *Emerging Topics in Artificial Intelligence Selected from IEA/AIE2021*
Guest Editors: Ali Selamat and Jerry Chun-Wei Lin

✉ Marcin Kawalerowicz
marcin@kawalerowicz.net

Lech Madeyski
Lech.Madeyski@pwr.edu.pl

[1] CODEFUSION Sp. z o.o.,
Armii Krajowej 16, Opole, 45071, Poland

[2] Faculty of Electrical Engineering, Automatic Control and Informatics, Opole University of Technology, Proszkowska 76, Opole, 45272, Poland

[3] Department of Applied Informatics, Wroclaw University of Science and Technology, Wyb.Wyspianskiego 27, Wroclaw, 50370, Poland

2. extending the experiment execution section with instrumentation, metrics and model creation description,
3. extending the analysis section by performing randomisation tests techniques,
4. preparing a new prediction models and comparing the original prediction results with results obtained from the new models,
5. assessing the acceptance of the proposed CBOP practice in real software development settings via a survey inspired by the Technology Acceptance Model (TAM).

This work tests the CBOP idea in an industrial setting and assesses its acceptance among professional software developers. We hope that this evaluation will contribute to a wider discussion of the topic within the software development community.

2 Background

SDP is a mature field [14, 29, 33, 44, 59]. It is seen as a cost-effective [2] way to detect problems that arise from a software defect. Some researchers investigated just-in-time SDP approaches [30, 60] and called it just-in-time quality assurance [4] or just-in-time defect identification [59]. There are several tools available to support just-in-time SDP, such as CC2Vec [24] and DeepJIT [25]. There are also older tools, where SDP is performed live in the Integrated Development Environment (IDE) described and prototyped in [33, 42]. The approach we proposed in [31, 32, 41] builds on top of those research (especially using the IDE approach) and combines it with prediction of software build outcomes using source code [21] and CI. The build result on the CI server can be seen as an unambiguous indication of a bug-introducing change that can be combined with metrics derived from the code that was used during the build.

This is different from a popular approach in defect prediction that determines code functionality based on lexical examination of the commit message or information from bug tracking software [1]. CBOP uses the real CI server build information to classify the change as buggy or clean. The assumption is simple: if the build fails, there is something wrong with the code and should be classified as "buggy."

In CBOP, we use machine learning to create a prediction model based on the CI build result and the metrics derived from the source code repository. The "continuous" notion in CBOP is achieved in two ways:

1. by implementing real-time feedback for the developer in his development environment,
2. by enclosing the build and learn process in an uninterrupted loop.

The real-time developer feedback is ensured by tools that work on the developer machine, preferably directly in his IDE. The changes he or she is introducing to the source code are constantly monitored. The metrics for the changed parts of the code are calculated and sent to the prediction model that classifies it as problematic or not. The prediction of whether the code will build is then displayed to the developer in his IDE using an extension or an add-on, so he or she can know to fix the problematic code.

The CBOP process is depicted on the diagram showed in Fig. 1. We see the developer interacting with his IDE and committing changes to the source code repository. The changes are finally pushed to the central repository and the CI server starts the build process. The build results are used together with the metrics derived from the repository to repeatedly re-trained the prediction model. This model is then used by the developer in his IDE to predict further build results.

3 Experiment plan

In this section, we describe the preparation of the experiment. While scoping, we state the goal of the experiment. Then we select the context and formulate the hypothesis. The preparation is concluded by variable and subject selection.

3.1 Scoping

The goal of the experiments are as follows [57]:
Analyse the CBOP practice
for the purpose of evaluation of the practice
with respect to its effectiveness to reduce the number of failed builds
from the point of view of the researcher and project manager
in the context of a small group of professional software developers
(subjects) working on an industrial-grade software project (object).

In other words, we want to determine whether the new CBOP practice will reduce the rate of failed builds in day-to-day work on an industrial-grade software project. According to the terminology used by Wohlin et al. [57], this is a "multi-test within object study". Although the software project and the subjects (i.e., the software developers) will not be chosen randomly, we do use randomization techniques in our experiment.

To perform the experiment, a set of software tools called Jaskier was created. The toolset was described in [32]. A set of metrics for two real business-driven software projects was harvested using a source code repository and a database

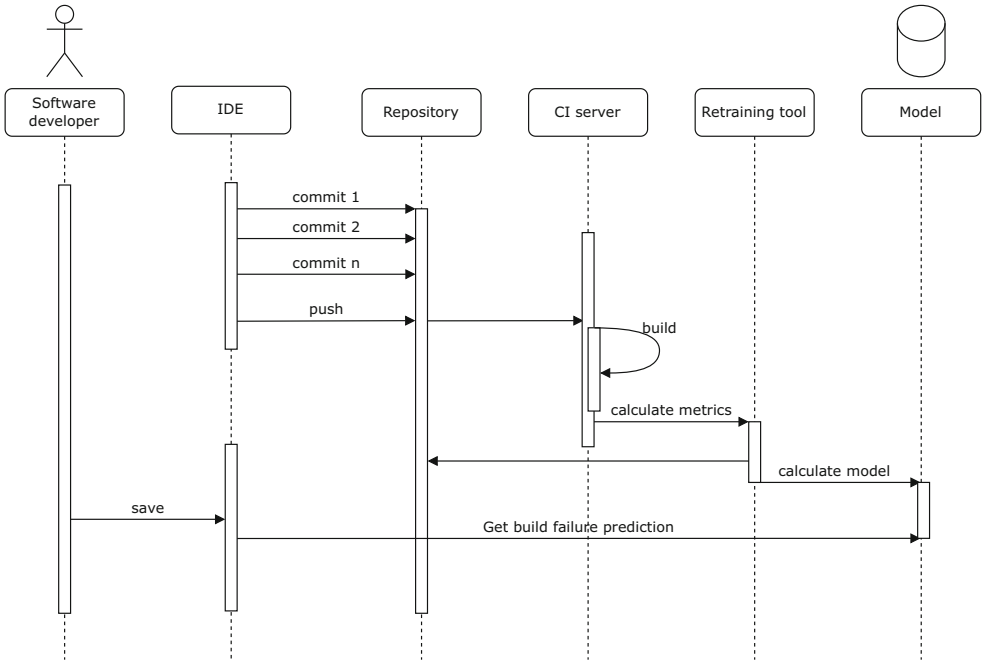


Fig. 1 Continuous build outcome prediction diagram

containing the build process outputs for those repositories. The metrics are described in Section 4.2.

3.2 Context selection

Two real ongoing software projects were selected. The first, termed ProjDotNet, is a Microsoft .NET based project in its 4th year of development. The second, termed ProjWeb, is a web-based Angular client (ProjWeb) for banks and leasing companies that uses ProjDotNet as an application programming interface (API).

Table 1 shows some data about the ProjDotNet and ProjWeb projects derived from it at the end of June 2018.

According to the classification by Wohlin et al. [57], the context of the experiment can be summarised as: an on-line (in vivo) project, involving professional developers, dealing with a specific, real (not toy) development problem.

The CBOP practice was not introduced to the project until the start of the experiment. There was neither defect prediction nor build failure prediction used in this project from its beginning. Build results were recorded started on 2018-06-20 until 2018-10-02.

Table 1 Project details

Number of distinct committers	18
Actively working in project	6-8 developers/week
The total number of commits	over 10 000
Total number of files	over 9 000
Total Lines of Code	over 4 500 000
Average build commits per active week	approx. 60

3.3 Hypothesis formulation

We formulate the following hypotheses:

H0 (null hypothesis): Using the CBOP practice does not influence the Failed Build Ratio (FBR).

H1: Using CBOP reduces FBR.

3.4 Variables selection

In the experiment, we control the time when we apply CBOP or not, so CBOP is an independent variable in our design. Specifically, we make predictions available to the developers only at certain randomized times across the development.

We monitor the build failure count and use it to calculate FBR. Every push to the central repository ends with the build on the CI server. The possible results one may observe on Jenkins CI are: `SUCCESS`, `FAILURE`, `UNSTABLE`, `ABORTED`, `NOT_BUILD`.[1] Only the results of type `FAILURE` are counted as failed builds. We do not count `UNSTABLE` builds as failures, as they are not necessarily associated with failures but, e.g., bad smells in production code or tests. The `ABORTED` and `NOT_BUILD`

[1] Jenkins CI Build results: http://javadoc.jenkins-ci.org/hudson/model/Result.html.

are not counted as failures as well. As a result, we calculate the failed build ratio (FBR) as follows: $FBR = \frac{\#FAILURE}{(\#UNSTABLE+\#ABORTED+\#NOT_BUILD+\#SUCCESS+\#FAILURE)}$.

3.5 Selection of subjects

The experiment will be conducted on a sample of the population of software developers. The company in which we conduct the experiment is a Poland based subcontractor of a larger software development company, which is based in Germany. The subcontractor leases four resources to its partner. One resource is 160 hours of work in one calendar month. It amounts to 640 hours of work (all of them not necessarily working full hours on that specific project). The work is carried out by 5-7 people. Four to five software developers, software tester, and a user experience specialist. All developers involved in the project are professionals with a master or bachelor degree in computer science with various experience levels: from 1 to 5 years of industrial experience as a software developer. The main task of all developers described here is the work in the subcontracted project described in detail in Section 3.2.

On the contractor side, there are four to five developers and one intern involved in that project. The developers are working full-time on that project, while the intern is working only part-time. One of the developers on the contractor is a software architect with 5+ years of experience, and the three remaining developers are juniors with 1 to 3 years of experience.

Because we are interested in the largest possible sample of subjects, we used a convenience sampling technique to select the subjects—we selected all available subjects involved in the project. For the planning phase of the experiment, we used a sample size of four for all calculations. We were sure that we will be able to conduct the experiment with at least four developers (working on the project on the subcontractor side). Before the start of the experiment, we were able to incorporate additional developers on the contractor side (the number of participants was 9).

3.6 Choosing experiment design type

In essence, we have here an experiment with one factor (the use of CBOP) and two treatments (presence or absence of the prediction). The participation of four developers from the contractor side in the experiment was certain. This rather small number of participants led us to use a small-n experimental design to study the effect of CBOP.

Dugard et al. [17] provided a list of experimental designs for single-case and small-experiments, see Table 2. Because we have expected at least four participants, we have not considered any of the "single-case" designs provided by Dugard et al. [17]. Careful examination of other conditions for those experiments enabled us to find a proper design for our situation ("A small-n repeated-measures design with two conditions and replicates"):

1. Do we have at least two participants? Yes.
2. Do we have two conditions to compare? Yes. (CBOPoff - No build outcome prediction available for the developer, CBOPon - build outcome prediction available for the developer)
3. Will each of the participants receive each of the conditions on at least two occasions? Yes.
4. Is it possible to assign conditions in random order? Yes.

Choosing the right period to alternate between the phases was not an easy task. We have gathered some actual data from 6 weeks preceding the start of the experiments on the rate of the various build results to determine the phase alternation interval. Based on a measured average of 54 builds per week, with six failed builds per week, we selected a period of one day to alternate between phases.

Next, we have proceeded to calculate the possible power of the experiment. We expected to have at least four participants in our team and four weeks to perform the experiment, giving us 20 observation days (assuming five working days per week). Before the experiment started, and during the preparation to the experiment, it turned possible to obtain more participants by incorporating software developers working at the customer and to double the possible experiment duration. In order to estimate the power of our experiment, we used a power function for an equivalent large-n design [17]. Lacking previous experiments in that field, we estimated effect size at 0.5 of a standard deviation difference between means. The α level was set conservatively to 0.05. The pwr R package was used[2] to calculate power for two groups and 20 observations as 0.87.

4 Experimental environment

This section describes technical experiment details. We present the instrumentation devised for this experiment. The metrics used to create the prediction model, and the model itself. We conclude with a short description of software tools used by the developers for CBOP.

4.1 Experiment instrumentation

The experimental architecture is depicted in Fig. 2.

Build results and associated commits are stored in an on-premise Bitbucket[3] database. The model was trained on build results, commit hashes, and metrics from the

[2] https://www.rdocumentation.org/packages/pwr/versions/1.3-0

[3] https://bitbucket.org/

Table 2 Choosing the experiment design after [17]

Types of experimental design	No. of participants	No. of observation occasions	No. of conditions	Experiment design
ANOVA analogous	1 participant	Several equivalent occasions	2 conditions	A single-case one-way design
ANOVA analogous	1 participant	Several occasions falling into two or more groups	2 conditions	A single-case randomized blocks design
ANOVA analogous	1 participant	Several occasions	2 factors with 2 levels each	A two-way factorial single-case design
ANOVA analogous	2 or more participants	1 occasion	2 conditions	A small-n one-way design
ANOVA analogous	2 or more participants	One occasion for every condition	2 conditions	A small-n repeated measures design
ANOVA analogous	2 or more participants	Several occasions to every condition	2 conditions	A small-n repeated measures design with replicates
ANOVA analogous	Multiple of 4 participants	1 occasion	2 factors with 2 levels each	A two-way factorial small-n design
Phase design	1 participant	Series of observations before and after intervention	2 conditions	The single-case AB design
Phase design	1 participant	Series of observations before, during and after intervention	2 conditions	The single-case ABA design
Phase design	2 or more participants	Series of observations before and after intervention	2 conditions	The multiple baseline AB design
Phase design	2 or more participants	Series of observations before, during and after intervention	2 conditions	The multiple baseline ABA design

Bitbucket database. Training data was gathered into a Microsoft SQL Server database and provided to the model in the Azure Machine Learning cloud. We hosted the final model as a web service and connected to this service from Visual Studio 2017 IDE using our custom-made extension. Normally the Jenkins server infrastructure can enable the model to continuously learn from new data; however, to avoid a reciprocal relationship between the prediction model and the actions of the developers that might influence the subsequent model [36], we turned off the ability of the system to continuously learn from the CI for the time of the experiment. We did not want the actions of developers made under the influence of the predictions to change the model during the experiment.

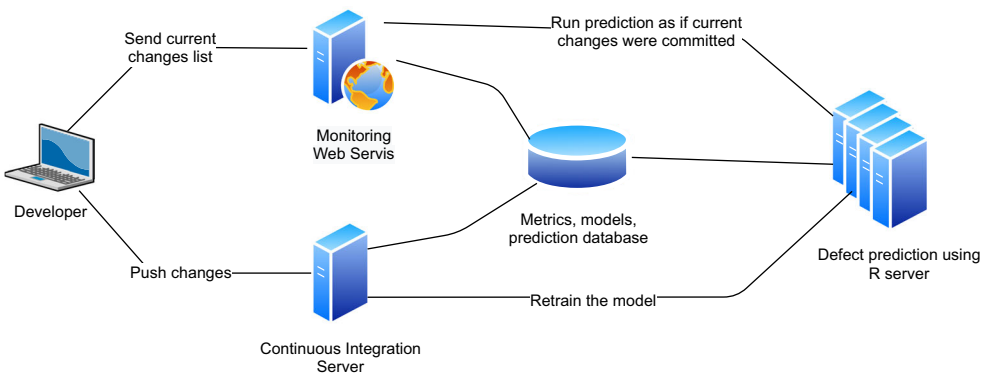


Fig. 2 Continuous build outcome prediction pilot project architecture

4.2 Metrics

Our software mines the Git repository to search for relevant metrics. For every build commit (being the commit that triggers the build on the CI server), all relevant file changes are determined. Those are all changes committed (possibly in a series of subsequent commits) to the repository since the last build commit. All changed files are determined and the metrics for those files are mined and calculated. Those metrics serve as predictors in the build outcome prediction model. The metrics are partly inspired by Madeyski and Jureczko [40] that validated the metrics for defect prediction on a large set of projects. Conceptually the defect prediction is very close to CBOP therefore, in addition to references relating to metrics that are directly applicable to CBOP, we are also inspired by references relating to the metrics for defect prediction. At the time of writing this paper, our software mines the following metrics.

1. NumberOfRevisions (NOR)
2. NumberOfDistinctCommitters (NODC)
3. NumberOfModifiedLines (NOML)
4. TotalNumberOfRevisions (TNOR)
5. BuildDateTimeLocal (BDTL)
6. BuildCommitDateTimeLocal (BCDTL)
7. LastBuildStatus (LBS)
8. CommitAuthorIdentification (AI)

Number of revisions (NOR): a count of revisions of a given file since the last build. It is the number of commits on all branches that took part in the build. The reasoning for including this metric into our set is that the more revisions pass since the last build, the more likely it seems to be that the build will fail [45].

Number of distinct committers (NODC): a count of all developers involved in changes in a given file since the last build. Every developer is counted only once, no matter how many revisions he or she made. The reasoning behind including this metric in the model is that the more distinct committers involved in code changes, the more likely they made some uncoordinated changes that can lead to the build failing. As the experiment described in [56] showed, this metric can yield only a slight improvement in the prediction result, but more recent studies like [27] play a significant role in the change of defect state.

Number of modified lines (NOML): a simple metric that contains the number of lines changed in a given file since the last build. It contains the cumulative number of changed lines in all revisions of the file since the last build. The reasoning for including this metric is that the more changes in a given file, the more likely the build breaks [61].

Total number of revisions (TNOR): a count of all changes in a given file since the beginning of the repository. It contains the number of all revisions of this file, no matter which developer committed the file. The reasoning for including this metric into consideration is that the more revisions, the bigger the "entropy" and the more likely the build will fail [14].

Local date and time of the build (BDTL) and commit (BCDTL): metrics containing the date and time of the build itself and the commit that caused the build. Both timestamps are stored as local time. The builds triggered in the last minutes of a work day are known in the developers' community to sometimes be very problematic. Developers are often pushing not ready changes into the repository shortly before going home. Sometimes it is even considered a desired behaviour to know where to start on the next day. As far as we know, this is the first usage of a similar metric for such purposes.

Last build status (LBS): whether the previous build was a fail or a success. It is often the case that fails come in series. Meaning that if we have one failing build, it is often the case the developers need more than one build to stabilize the build and make it pass (succeed). This metric is a variation of another well known software metric called Number of Defects in Previous Version (NDPV) [19, 40].

Author identification (AI): it is a well-known, although often silenced, fact that some developers are causing more problems to the builds (while integrating software) than others. Some researchers are going so far as to create separate defect prediction models for individual developers [18, 28]. The AI contains a list of anonymized identifiers of the authors of the changes since the last build.

Alongside the metrics, a number of other information is stored. The following data is stored in database:

1. Build result
2. Project name
3. Path
4. Commit hash
5. Build commit hash
6. Export date

The build result is the dependent variable in our build outcome prediction model. It classifies the changes associated with metrics as buggy or clean. It is obtained from the CI server. "Project name" is the name of the software project. "Path" contains the path and the file name associated with the metrics. "Commit hash" is the repository commit identifier and "build commit hash" is the repository commit that caused the build on the CI server. "Export date" is the time stamp of the data record associated with the metrics and additional data.

Our software gathers the metrics from a Git repository. We have written a C# program that calculates the metrics and stores them in a Microsoft SQL Server database. The data from the database is used to create the classification model. The model is created using R code or Azure ML.

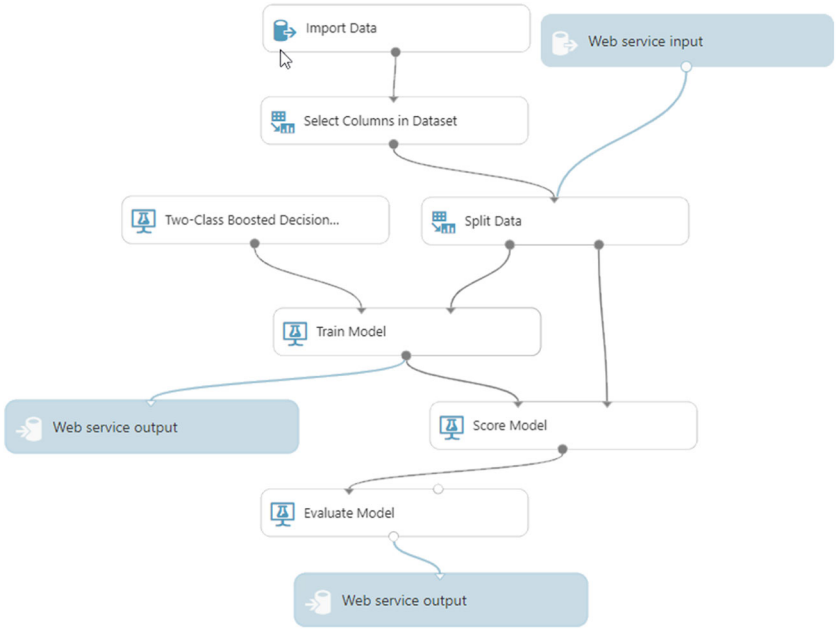


Fig. 3 Azure ML training experiment

Models can be created within Azure ML using experiments, which are roughly equivalent to R scripts but are created using a simple diagramming tool.

The metrics gathering program, we created, has a form of a simple command-line tool that can be used locally or as a part of the CI process. The tool needs a path to a local Git repository, a source of the buggy/clean information (for example, Bitbucket database), and a database to store the metrics. If the metrics database is empty, it pulls all available history and generates the metrics for all builds. If there are data in the database from the previous tool runs, only the metrics for the newer builds are calculated. The simplest way to use the metrics gathering tool is to automate it as part of the CI process. For example, in a post-build CI job. First, an ordinary CI job is run on a software project. After that, a chaining mechanism of the CI server is used to run a new CI event that contains the data gathering tool (and possibly the retraining of the classification model).

4.3 Model

The classification model creation could be a continuous process. Depending on how often the build is triggered on the CI server and how big the classification model is (in terms of size and time needed to create it), the retraining R process or Azure ML experiment can be started after every CI build or using a more conservative schedule (for example as a part of the "Nightly Build").

We are aware that the success of our tool (and practice) relies very much on the quality of our prediction algorithms. Therefore, the decision to abstract the creation and use of the model as far as possible. It is hidden behind a custom-made web service, so the actual model creation can be switched without changing the client part. The intention was not to focus at the beginning on the search for the optimal classification algorithm, rather than to make it possible to adopt different algorithms without a problem in the future. The technical ways to create the model are:

1. use independent R script run on a local R installation,
2. use an R script on the Microsoft R Server,[4]
3. use Azure ML - a cloud R based machine learning infrastructure from Microsoft.[5]

Because we had a distributed team taking part in the experiment, we used Azure ML infrastructure during the evaluation of CBOP. The so-called "Azure ML experiment", that creates the model, was described in detail in [32]. The "Azure ML experiment" has a form of visual diagram, where the nodes are individual steps performed to create and score the model. The Azure ML experiment, as it is designed in Azure cloud, is shown in Fig. 3. First the data for the model is imported from a database, then columns with features are selected. The dataset is split into a training set and scoring set. The model is trained using a random forest algorithm from [12]. At the end of the algorithm, simple scoring and evaluation was performed.

To score the model outside Azure ML we used k-fold cross validation and the area under the ROC curve [53]. Statistics associated with the confusion matrix [35] were calculated (ACC, MCC and F1) and plotted (sensitivity,

[4] https://www.microsoft.com/en-us/cloud-platform/r-server

[5] https://azure.microsoft.com/en-us/services/machine-learning-studio/

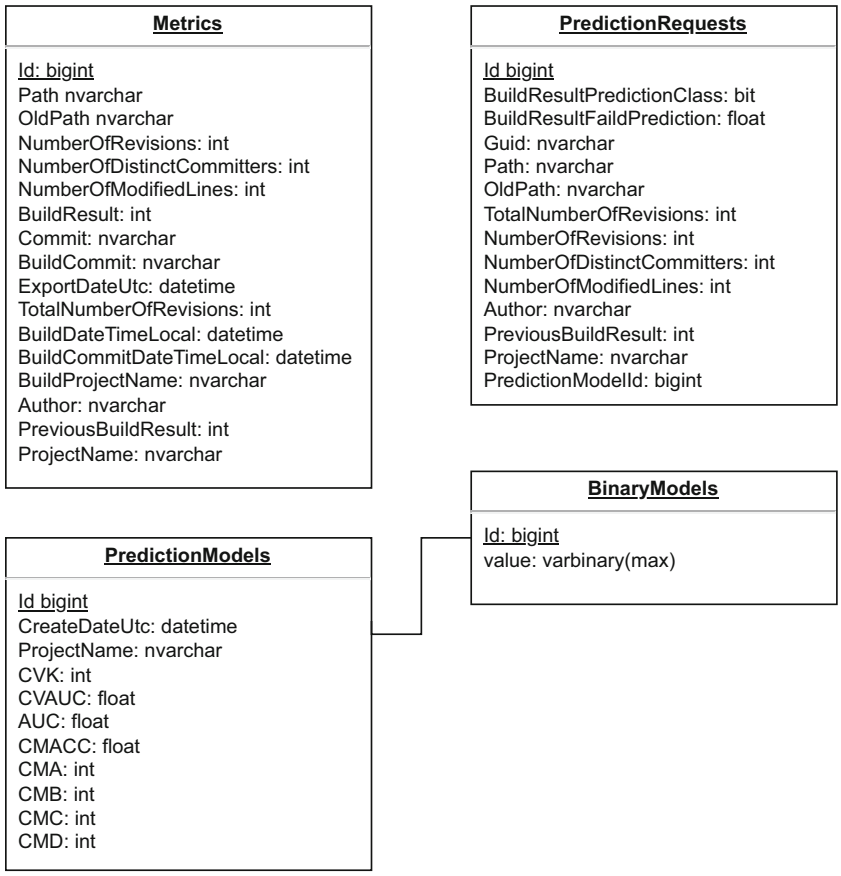


Fig. 4 Metrics, classification models and prediction request database

specificity). For our project setup, we reviewed several classification methods using R [26]. We used decision trees extensively on our development machines, for simplicity and performance. We were using rpart package [8] to create our model (the implementation is based on the book [7]). On the server side, we are using more resources and a time-consuming random forest algorithm (randomForest [37] based on [6]). In the Azure ML cloud, the random forest implementation from [13] is used. The classification models are stored in the database. Figure 4 shows the prediction database architecture.

4.4 Clients

At the time of performing the experiment, we had three prediction clients implemented:

1. generic command line tool (Microsoft .NET based),
2. Microsoft Visual Studio Extension,
3. VSCode Extension.

All clients watch the software project directory and detect a change in a file inside this project. If a change is detected, the metrics are calculated for the current state of the project. The prediction client connects the prediction service and sends the metrics. The prediction service writes the data to the SQL Server database and, in this experiment setup, executes Azure ML service over our custom-made web service. The prediction (label and probability of fail) is then written back to the database and the control is returned to the web service. The web service returns the data to the prediction client, where it is presented to the developer.

5 Experiment results

5.1 Validity evaluation

In accordance with Wohlin et al. [57], we discuss threats to the validity of our research.

The threat to the internal validity that applies to our research is that a subject may react differently as time passes (maturation). We think this threat is to a large extent addressed by the CBOP on (aka CBOPon) / off (aka CBOPoff) randomized assignment, see Table 3.

Another important threat is selection threat, meaning that the group selected for the experiment will not be representative for the larger population. We think that this threat is at least partially addressed by the fact that the subjects in the experiment are from two different companies in two different countries (Poland and Germany). We might

Table 3 CBOPoff (A) / CBOPon (B) assignments and total results for individual developers

Developer	Assignment plan	Days in exp.	Obs. count	Total builds	A total builds	A failed builds	B total builds	B failed builds
Developer 1	AAAAAABBBBBBBBBBBBB BBBBAAAAABBBBBBBBBB	44	33	63	20	3	43	13
Developer 2	AAAAAAABBBBBBBBBBBB BBBBAAAAAAAAABBBBBB	27	20	32	7	0	25	3
Developer 3	AAAAAAAABBBBBBBBBBB BBBBAAAAAAAAABBBBBB	37	27	47	22	0	25	1
Developer 4	AAAAAAAAAABBBBBBBAAA AAAAAAAABBBBBBBBBBB	60	84	227	134	43	93	15
Developer 5	AAAAAAAAAAABBBBBBBBB BBBAAAAAABBBBBBBBBB	43	29	65	24	0	41	1
Developer 6	AAAAAAAAAAAABBBBBBBB BBAAAAAAAABBBBBBBBB	54	61	134	58	21	76	18
Developer 7	AAAAABBBBBBBAAAAAAA AAAAABBBBBBBBBBBBBB	35	50	124	51	7	73	20
Developer 8	AAAAAAAAABBBBBBBBAAA AAAABBBBBBBBBBBBBBB	40	55	239	94	9	145	50
Developer 9	AAAAABBBBBBBBBBBBBB BBBAAAAAAABBBBBBBBB	46	51	139	27	2	112	24

also face the ambiguity of the direction of causal influence. The fact that the developers are getting the build outcome prediction might in fact make them less cautious about their commit.

Threats related to the problem of generalising the research results to a wider population of software developers are threats to external validity. The threat of "interaction of selection and treatment" is addressed to some extent by the fact that in both teams there was a wide distribution of experience and knowledge (from juniors to seniors) and good diversity in culture and educational background (two countries; different education paths). The threat of "interaction of setting and treatment" is mitigated through the use of two technically different projects (.NET and Angular), the usage of industry-standard tools (Visual Studio, VSCode), and a real, not toy-like, project. However, this threat needs to be considered because technical differences between other types of projects might impact the ability to generalise the results. We conducted the experiment during a period of several days to mitigate the threat of "interaction of history and treatment". The time of day should not impede the observations. The research was conducted solely in a real-life software development environment, which makes the external validity considerations much less critical.

Another type of threat we considered is a construct validity threat. This threat concerns generalising the results of the experiment to the ideas behind it. To address "inadequate preoperational explication of constructs", we have defined, as clearly as possible, what we are looking for - fewer failed builds. We hope to widen the generalisation to a broader concept of defect (not only failing build) prediction, but we decided to start with the narrower problem of broken builds, which developers have to deal with most often. The other threat that we think needs to be considered is "restricted generalisability across constructs". It is important to check that the approach we are proposing will not affect the project negatively. For example, the prediction itself might give the developers false beliefs and thus result in more careless commits. We have a similar concern regarding social threats to the experiment. It is possible that the behaviour of the software developers will change due to the usage of our CBOP tools. They may feel more secure while receiving positive predictions, and thus they would more boldly commit insecure changes to the repository.

The last type of threat we have considered is the threat to conclusion validity, which affects our ability to draw correct conclusions about the influence of CBOP on FBR in a software development project. An important threat is the potentially low statistical power of the experiment (because of the low number of participants involved in the study). We tried to mitigate this threat by involving as many developers as possible and using randomization techniqes. Finally, we were able to use data from nine individual developers. The p-values are provided in Table 5.

5.2 Analysis

In the experiment, we analysed 310 project days worth of data coming from a total of nine developers. Table 3 shows the assignments for individual developers. The experiment

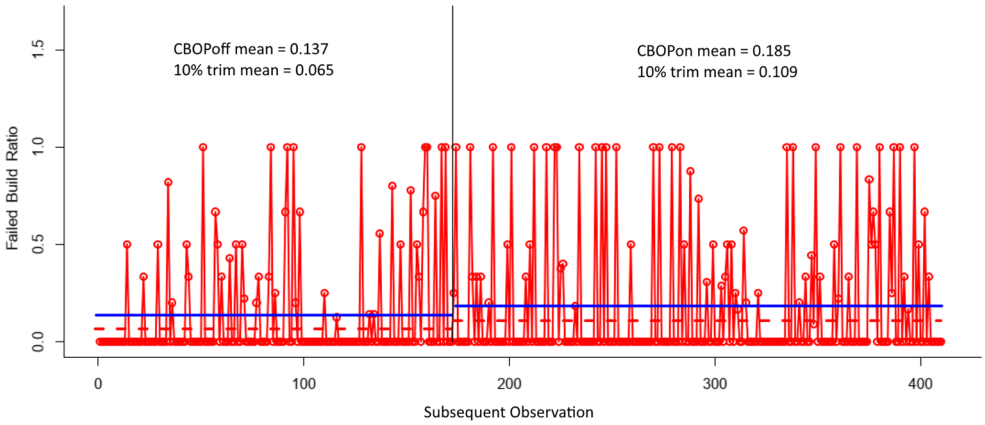


Fig. 5 Failed Build Ratio (FBR)

assignments were prepared 40 days in advance. Different developers worked on the project for a different number of days (because of sick days, vacations, different project assignments, etc.). Some days, the developers did not make any commits, resulting in a build. Those days were omitted from the results. For those days, the developers were assigned a phase. If a developer participated in the experiment for longer than 40 days, the sequence started from the beginning. If a developer participated for fewer than 40 days, then only the days in which he or she participated were taken into account. The developers were working simultaneously in Visual Studio on the back-end .NET services and in VSCode working on the Angular front web client for the services. The results were aggregated for both projects separately.

5.2.1 Analysis of descriptive statistics

In total, CBOP was turned off during 173 days and turned on during 237 days of the experiment. The FBR dependent variable, calculated as described in Section 3.4, is plotted in Fig. 5.

We see an increase in the absolute FBR when the CBOP was turned on, from 0.137 (CBOPoff) to 0.185 (CBOPon). Figure 7 shows the box plot for the same data. The median in both phases was 0.

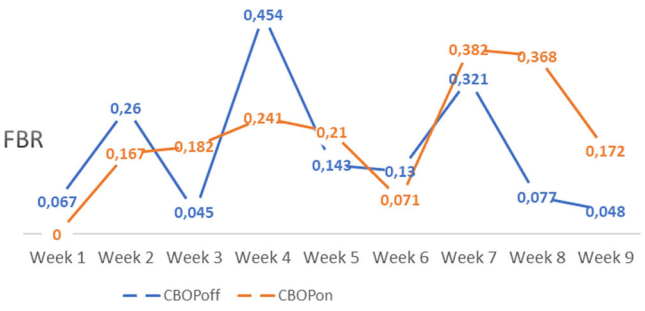


Fig. 6 FBR grouped by week for all participants

Because we had a rather small number of builds each day, we also calculated the averages of CBOPon and CBOPof for all developers grouped by the week of the experiment. Those values were plotted in Fig. 6.

Because variances were not equal (F = 0.70099, df1 = 172, df2 = 236, p-value = 0.01366), we may use Welch's t-test, but the null hypothesis cannot be rejected (t = -1.5579, df = 400.34, p-value = 0.12). Unfortunately, the data were autocorrelated in the CBOPon phase and had a visible trend in the CBOPoff phase. In such a situation, transformation using differencing is recommended. Unfortunately, it did not remove autocorrelation. Hence, Welch's t-test may not be reliable. We proceed further by analysing a robust measure of central location and effect sizes (Fig. 7).

Mean is not a robust measure of central location, and it can be strongly influenced by outliers, especially when the number of observations is small. However, the 10% trimmed mean (which is a more robust measure of central tendency) did increase from 0.065 (when CBOPoff) to 0.109 (when CBOPon). The standard deviation increased similarly from 0.281 (when CBOPoff) to 0.335 (when CBOPon). We will discuss possible explanations of such behaviour in Section 7, but now we focus on the size of the observed effect. Table 4 contains descriptive statistics for all phases and developers.

5.2.2 Analysis using randomisation tests

Parametric t-tests are controversial when it comes to multiple-baseline studies [9]. Especially in the CBOP case, where there is autocorrelation in the results, a t-test can be biased. Therefore, a good alternative is a randomisation test. Such tests have another advantage when there is a visible trend in the data. Randomisation tests do not invalidate the results for such data.

While preparing the experiment, the ABAB experiment plan was randomly generated (the plan is presented in Table 3). During the analysis based on the randomization

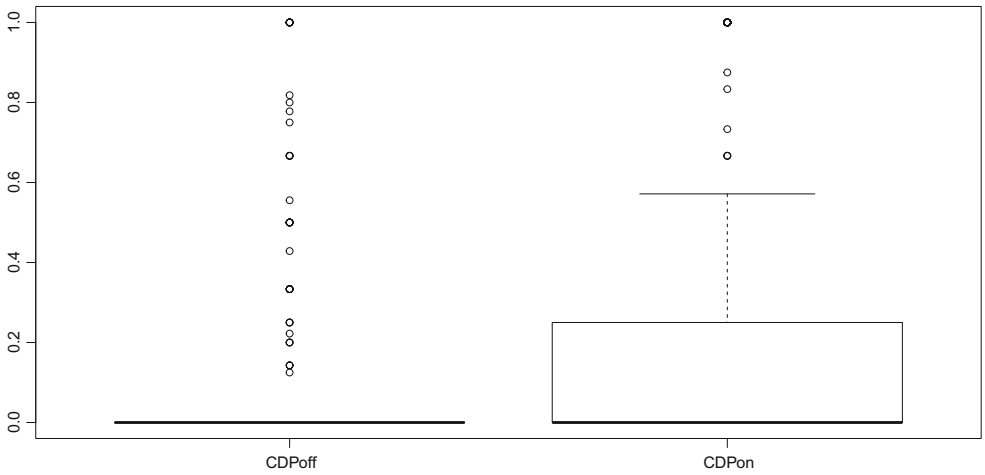


Fig. 7 Failed build ratio (y-axis) box plot

tests, a permutation of the random assignment is used to calculate the test statistic. Either a complete distribution of all possible assignments is generated and the test statistic is calculated for all the permutations, or a random sampling is used to generate a non-exhaustive randomization distribution if there is a large number of possible arrangements (thus making it computationally extensive). The test statistic is the located in the distribution and the p-value of the test is the proportion of the randomized statistics exceeding the observed statistic. A predefined significance level α is used and compared with the p-value. If the p-value is less than or equal to α the null hypothesis is rejected.

For the CBOP experiment, a complete distribution of all possible arrangements was generated, and the test statistic for this arrangement was calculated. The test statistic used for the randomisation test was defined as a difference in 10% trimmed means (μ) of the measurements between the phases A and B:

$$T = \mu_A - \mu_B \tag{1}$$

Because the ABAB experiment plan consisted of consecutive A and B phases, which can be enumerated as A1, B1, A2, and B2, the equivalent test statistic can also be defined as the difference between the sum of the means for the A ($\mu_{A_1} + \mu_{A_2}$) and B phases ($\mu_{B_1} + \mu_{B_2}$):

$$T = (\mu_{A_1} + \mu_{A_2}) - (\mu_{B_1} + \mu_{B_2}) \tag{2}$$

Onghena [46] enumerated a comprehensive list of various test statistics to be used in various situations including statistics for decaying and delayed effects, test statistics corrected for trends, and even for slope or for specific linear or nonlinear trends. He also stated that "any test statistic that is sensitive to the expected effect of the treatment is acceptable". The statistic used here is a straightforward statistic that corresponded with H1. The hypothesis was that the usage of CBOP would reduce FBR, so it was expected that the scores in phase B would be lower than the scores in phase A. That is why the expected lower μ_B was subtracted from the expected higher μ_A. The usage of trimmed means aimed to eliminate the influence of outliers in the data. In order not to exclude

Table 4 Descriptive statistics for each developer for CBOPoff / CBOPon

	Obs. count		Mean		10% Trim mean		Median		SD	
	off	on	off	on	off	on	off	on	off	on
Developer 1	12	21	0.194	0.337	0.133	0.298	0.000	0.000	0.388	0.447
Developer 2	5	15	0.000	0.133	0.000	0.077	0.000	0.000	0.123	0.352
Developer 3	15	12	0.000	0.021	0.000	0.000	0.000	0.000	0.000	0.072
Developer 4	43	41	0.216	0.118	0.168	0.069	0.000	0.000	0.290	0.233
Developer 5	12	17	0.000	0.020	0.000	0.000	0.000	0.000	0.000	0.081
Developer 6	29	32	0.254	0.269	0.215	0.216	0.000	0.000	0.380	0.414
Developer 7	22	28	0.102	0.221	0.032	0.175	0.000	0.000	0.257	0.364
Developer 8	24	31	0.044	0.214	0.013	0.146	0.000	0.000	0.123	0.350
Developer 9	11	40	0.136	0.197	0.056	0.121	0.000	0.000	0.323	0.325

too many additional observations, the 10% trimmed mean threshold was used. The amount of obtained data points was influenced by the fact that some developers did not commit the source code to trigger the builds on a given day, which resulted in missing data for those days.

The observed statistics were calculated according to the plan shown in Table 3. The software that enabled experimentation on CBOP assigned a phase based on that plan. Every day that a developer launched the IDE, a new phase was assigned. It could have happened that the developer did not perform any commit causing the build to happen on that day. Therefore, we had missing data for that phase. When a developer was not working on the project, no assignment was made for that day. That is why we sometimes had fewer data points than assumed in the plan (see Table 3). When we had more data than in the plan, only the ABAB phases according to the plan were used for the calculation. For the randomisation test analysis, each developer was treated as a single case. The plots of the observations are shown in Fig. 8. The calculated observed statistics and the p-value calculated

Table 5 CBOP statistic and p-value for ABAB randomisation test for each developer

	t	p-value
Developer 1	−0.109	0.332
Developer 2	−0.083	0.477
Developer 3	−0.029	0.815
Developer 4	0.092	0.076
Developer 5	0.0	0.968
Developer 6	−0.061	0.839
Developer 7	−0.134	0.796
Developer 8	−0.205	0.826
Developer 9	−0.092	0.84

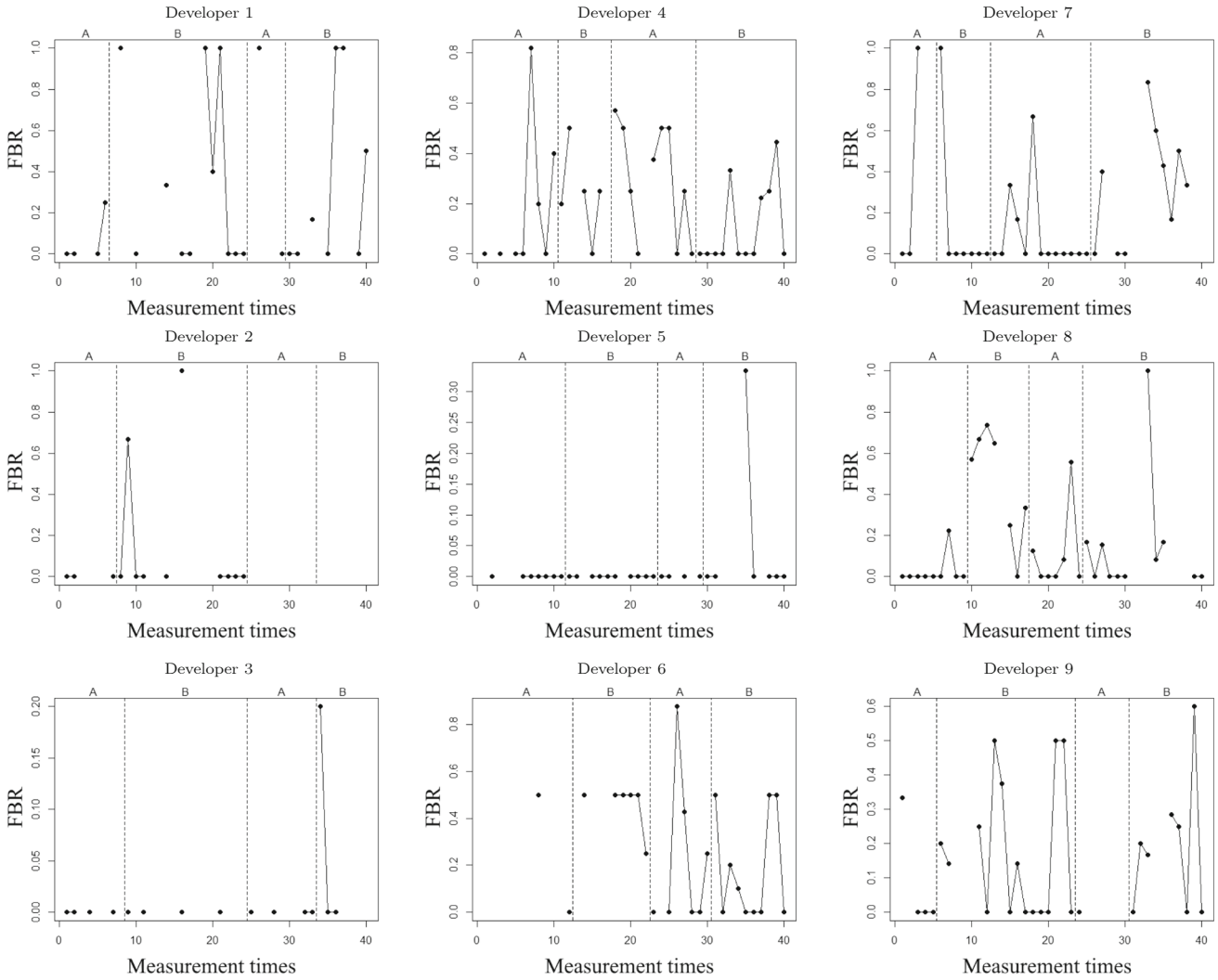


Fig. 8 CBOP experiment observations in ABAB design for individual developers

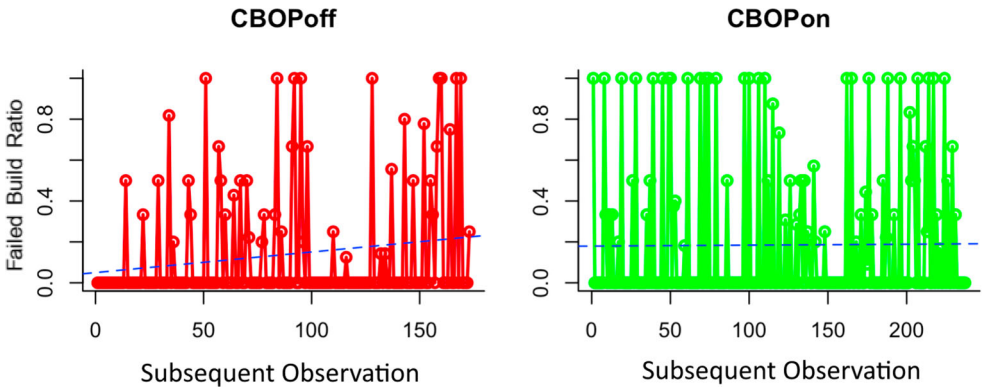


Fig. 9 Visualisation of trends

for the complete enumeration of all possible assignments are shown in Table 5. The results of this analysis did not permit rejection of the null hypothesis. For most of the developers, the observed statistic was contrary to expectation, suggesting an effect in the opposite direction.

5.2.3 Effect size

It is recommended practice to assess the magnitude of a treatment effect (CBOPon vs CBOPoff in our case) using effect size measures [20, 34, 39]. To grasp the effect of CBOPon vs CBOPoff, we reported the most common effect size measures:

- ES calculated as the difference between the intervention CBOPon and baseline CBOPoff means divided by the standard deviation of the baseline (ES=$\frac{M_{CBOPon}-M_{CBOPoff}}{SD_{CBOPoff}}$),
- d-index uses a pooled standard deviation and thus may be more appropriate when the variation between phases differs (d-index=$\frac{M_{CBOPon}-M_{CBOPoff}}{SD_{pool(CBOPoff,CBOPon)}}$).

Both values (ES=0.16923, d-index=0.15156) were below 0.87 and indicated a "small effect size". However, when there was a trend in any phase, then both measures were not appropriate, and we needed to use more sophisticated effect size measures such as g-index, PEM (the percentage of data points in the intervention phase (CBOPon) exceeding the median of the baseline phase (CBOPoff)) [38] and PAND (the percentage of all non-overlapping data) [47]. In our case, there was a visible trend in the baseline CBOPoff (see Fig. 9).

Hence, we reported and relied on the following effect size measures:

- $PEM = 0$ (i.e., lower than 0.5) can be interpreted as "not effective" according to Ma [38].
- $PAND = 0.578$ (i.e., lower than 0.69) can be interpreted as "debatable effectiveness" according to Parker et al. [47].

This indicated that CBOP was "not effective" or that its effectiveness was contrary to expectation and in the opposite direction (i.e., may have led to an increase in absolute FBR). However, the calculated p-values in Table 5 did not allow any statistically significant conclusions to be drawn.

6 Post-hoc experiment analyses

To develop the paper [31] further, we conducted two post-hoc analyses. First, we analysed the quality of the prediction model used within the experiment. Second, we described a TAM-inspired survey that was conducted among industry specialists to predict the usage of Jaskier. We asked the developers who took part in the experiment to answer the survey questions, and we analysed their answers in the same way. This section describes the results of these post-hoc analyses.

6.1 Model analysis

As described in Section 4.3, the model used in the experiment was created using Azure ML. To train the model, a two-class boosted decision tree [23] algorithm was used. The model accuracy calculated by Azure ML (ACC) was 0.873. The confusion matrix and ROC (receiver operating characteristic) curve are shown in Fig. 10. Because the quality of the model was one of our concerns, we decided to evaluate it based on the real build results obtained during the experiment and to create a new model using the newest version of the mlr3 library[6] to check whether we could obtain better prediction results.

During the experiment, we did not store all prediction data in the database. Only the prediction requests during the CBOPon phase were stored. The predictions were shown only to the developers in their IDE. To check whether the model created using the mlr3 library would perform

[6] https://mlr3.mlr-org.com/

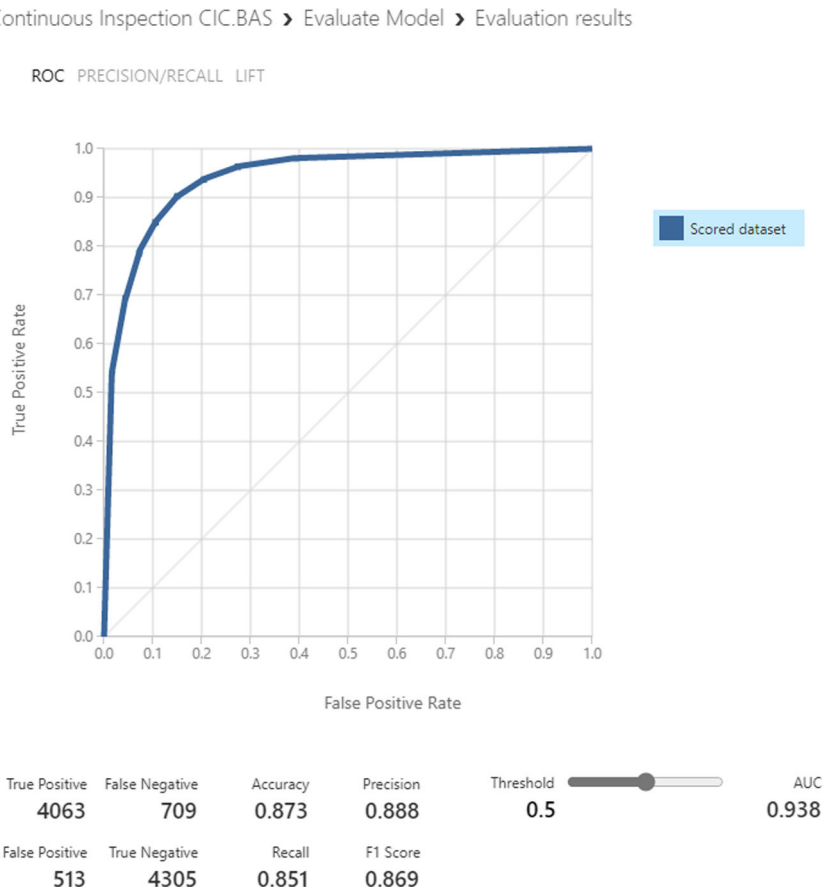


Fig. 10 Azure ML experiment evaluation results screenshot

better on real-life data, we needed to calculate the previous prediction based on the prediction requests. Using the same model as in the experiment, we recalculated the build predictions and matched them with the actual build results. We were able to match 981 predictions with the real-build results. Table 6 shows the confusion matrix for those matches in the original model area.

The new model was created using the mlr3 library and classif.ranger learner. It is a new and fast implementation of random forests [58]. The model was calculated using code presented in Listing 1. Data from the metrics database table were used only for the developers that took part in the experiment. Because we worked with unbalanced data (we had a lot more builds that ended in success than in failure), we tested SMOTE (Synthetic Minority Oversampling TEchnique) [10], but it did not improve the results. For the BDTL and BCDTL dates, we calculated the number of minutes that passed since midnight. To better compare the new models with the one used in experiment we did not use the nested cross-validation, but evaluated all models on the same test set. Resampling results (accuracy = 0.984, MCC = 0.956, F1 = 0.990) were high, but on the test set that the model never seen prediction results were lower (accuracy = 0.889, MCC = 0.656, F1 = 0.931). The confusion matrix with comparison to the original model is shown in Table 6.

The development of ML models to predict the CI build outcome is evolving in the direction of genetic programming [48–50]. This approach is especially helpful for dealing

Table 6 Confusion matrix comparison

	Original model (MCC=0.139) Predicted		classif.ranger (MCC=0.656) Predicted		DL-CIBuild (MCC=0.718) Predicted	
	Success	Fail	Success	Fail	Success	Fail
Actual						
Success	510	109	734	79	717	48
Fail	255	107	30	137	47	169

```
train_task = as_task_classif(
    train_set,
    target = "BuildResult",
    positive = "1"
)
test_task = as_task_classif(
    test_set,
    target = "BuildResult",
    positive = "1")
inner_resampling = rsmp("cv")
outer_resampling = rsmp("cv")
my_measure = msr("classif.mcc")
search_space = ps(
    num.trees = p_int(lower = 5, upper = 800, tags = "budget"),
    mtry.ratio = p_dbl(0.01, 0.95),
    min.node.size = p_int(lower = 1, upper = 3),
    replace = p_lgl(list(TRUE, FALSE)),
    sample.fraction = p_dbl(lower = 0.25, upper = 1)
  )
at = auto_tuner(
  method = "hyperband",
  learner = lrn("classif.ranger", predict_type = "prob"),
  search_space = search_space,
  resampling = inner_resampling,
  measure = my_measure,
  term_evals = 20
)
rr = resample(train_task, at, resampling = outer_resampling,
  store_models = TRUE)
rr$aggregate(my_measure)
at$train(train_task)
pred = at$predict(test_task)
autoplot(pred, type = "roc")
```

Listing 1 R source code snippet for classif.ranger model creation and tuning with mlr3

with the imbalanced distribution of failed and successful builds (as in our case). Saidani et al. in [51] introduced an approach for creating prediction models for CI build outcomes, which they called DL-CIBuild. The source code is freely available.[7] This approach uses long short-term memory (LSTM)-based recurrent neural networks (RNN). We used it to calculate a model based on our data, which reached an AUC of 0.903, MCC of 0.718, and F1 of 0.938. The confusion matrix for this model with comparison to the original model and the classif.ranger is shown in Table 6.

Interestingly, this new model calculated using mlr3 library performed better for the actual data than the old model. This was the case for the correct predictions as well as for the false positive and false negative predictions. The newest approaches in building prediction models for CBOP using genetic algorithms show a very promising development in the quality of prediction.

6.2 Using GQM to perform the TAM-inspired survey

Use of Microsoft Azure ML in the Jaskier project caught the attention of the Polish Microsoft branch in their so-called Corporate Territory. It is the field where Microsoft offers new solutions based on their products to corporate customers. For talks about the commercialisation of Jaskier as a tool for a wider audience, we used a goal question metric (GQM) to prepare and conduct a technology acceptance model (TAM)-inspired survey to check the acceptance among corporate customers.

According to GQM [3, 43], we defined a goal as follows:

Goal: To evaluate Jaskier's acceptance by professional software developers and specialists.

The questions and the metrics used to answer them were defined as follows:

Question 1: What is the perceived usefulness (PU) of Jaskier?

Metrics: better code actuator, efficient work actuator, action actuator, subjective PU.

Question 2: What is the perceived ease of use (EoU) of Jaskier?

Metrics: ease of idea comprehension, EoU, discoverability.

Question 3: Do the developers intend to use Jaskier?

Metrics: use intention, use prediction, commitment level, intention level.

Question 4: How is Jaskier perceived in terms of subjective norm (SN)?

Metrics: approval level, recommendation level, guidance level.

Question 5: How is Jaskier perceived in terms of organisational usefulness (OU)?

Metrics: success ratio, benefit ratio, enhancement ratio.

To give quantifiable answers to the GQM questions computing the sets of metrics), we designed a TAM survey that was based on a model introduced by [15, 16] and later extended by [55].

A presentation about Jaskier was given to four experienced corporate software developers. After the presentation, the TAM-inspired survey was performed. The same survey was used once again among the participants of the experiment and was conducted after the experiment ended. We gathered seven sets of answers from the developers who took part in the experiment.

Our TAM-inspired survey consisted of 17 questions divided into 5 groups. First, two groups were drawn from the first version of TAM [15]. Those groups were PU and perceived EoU. The answers to the questions in those groups were intended to clarify the intention to use (ItU) Jaskier among the respondents. The SN group was drawn from the second version of TAM [55]. Answers to the questions in this group described the perceived social influence on the intention to use Jaskier. Questions in the ItU group were selected to explain the direct intention to use Jaskier. Answers to the questions in the last group OU were determinants of the organisational impact of Jaskier perceived by the developers who took part in the survey.

[7] https://github.com/stilab-ets/DL-CIBuild/

To rate the answers, we used a bipolar 7-point Likert scale containing the following items:

1. Strongly disagree
2. Disagree
3. Disagree somewhat
4. Undecided
5. Agree somewhat
6. Agree
7. Strongly agree

The survey contained the following items (grouped by question and metric):

1. What is the PU of Jaskier?
   (a) Jaskier will help me/my team to create better code – better code actuator.
   (b) Jaskier will help me/my team to find bugs quicker – efficient work actuator.
   (c) Jaskier results are useful to act upon – action actuator.
   (d) I prefer traditional ways to detect errors and bugs in my software, and I do not find Jaskier useful – subjective PU.
2. What is the perceived EoU of Jaskier?
   (a) The idea behind Jaskier is straightforward and understandable – ease of idea comprehension.
   (b) Use of Jaskier extension is self-explanatory – EoU.
   (c) Jaskier results are easy to interpret and to act upon – discoverability.
3. What is the intention to use Jaskier?
   (a) If I had access to Jaskier, I would use it – use intention.
   (b) If I had access to Jaskier, I predict that I would use it – use prediction.
   (c) My personal level of commitment to using Jaskier is high – commitment level.
   (d) My personal intention to use Jaskier is high – intention level.
4. Subjective norm
   (a) Most people that are important in my professional career would approve of my use of Jaskier – approval level.
   (b) Most people that are important in my professional career would tend to encourage my use of Jaskier – recommendation level.
   (c) Most people that are important in my career would regard me as a better professional if I used Jaskier – guidance level.
5. Organisational usefulness
   (a) My use of Jaskier would make my organisation more successful – success ratio.
   (b) My use of Jaskier would be beneficial for my organisation – benefit ratio.
   (c) My use of Jaskier would enhance my work performance in my organisation – enhancement ratio.

Results of the survey are shown in Table 7. We focused on the central tendency measures and calculated the mode and median values along with variability measured by the first (Q1) and third (Q3) quartiles.

By transforming the answer to a negative "subjective PU" question, we see that Jaskier was seen as a rather useful tool, especially as an indicator for an action. Jaskier was perceived as very easy to use, with one important caveat: the results shown by Jaskier were considered moderately discoverable.

The original purpose of the survey was to predict the actual usage of Jaskier among the practitioners. As shown by Davis et al. [54], ItU correlated with the actual usage (which was not necessarily the case with PU and EoU). The answers of the corporate users and practitioners who took part in the experiment were aligned for PU and EoU but not for ItU. For ItU, the corporate users, who were shown only how Jaskier works but never used it in day-to-day work, showed considerably greater intention than the professional users who did use it. Both user groups perceived Jaskier rather ambivalently in regard to SN and rather positively in regard to OU.

7 Discussion

The results of the experiment indicate that during the CBOPon phase (in which the build result predictions were shown to the developers), the participants caused more failed builds than during the CBOPoff phase (where the results were not shown). The effect size was very small, but nevertheless the results indicate that the hypothesis about the positive influence of CBOP on FBR was not supported by the data. This surprising result led us to investigate more closely one of the threats we considered before the experiment: the "restricted generalisability across constructs" threat. We considered the possibility that the positive build prediction might give the developer false belief about the safety of the changes and prompt them to commit the code carelessly.

We found one explanation that is rarely considered in a software engineering context. The observed phenomena might have been caused by the human tendency to use

Table 7 TAM Survey results of Jaskier (CU=corporate user, PU=professional user, $\sum = CU + PU$)

Question	Mode			Median			Q1			Q3		
	CU	PU	$\sum$	CU	PU	$\sum$	CU	PU	$\sum$	CU	PU	$\sum$
Perceived usefulness:												
better code actuator	–	4	2	4.5	4	4	3.5	3	3	5.25	5	5
efficient work actuator	5	5	5	4.5	5	5	3.25	4	3.5	5	5	5
action actuator	6	6	6	5	5	5	3.75	3	3.5	6	6	6
subjective perceived usefulness	–	3	3	2.5	3	3	1.75	2.5	2	3.5	4.5	4
Perceived ease of use:												
ease of idea comprehension	6	6	6	6	6	6	5.5	5	5	6.25	6.5	6.5
ease of use	7	7	7	6	6	6	4.75	4.5	4.5	7	6.5	7
discoverability	4	3	3	4.5	5	5	4	3	3.5	5.5	6	6
Intention to use:												
use intention	–	6	2	5	4	4	3.5	2	2	6.25	6	6
use prediction	7	6	7	5.5	4	4	3.5	2	2	7	6	6.5
commitment level	–	6	6	4.5	2	3	2.5	1.5	1.5	6.25	5.5	6
intention level	6	6	6	4.5	4	4	2.5	3	2.5	6	5.5	6
Subjective norm:												
approval level	6	5	6	6	4	5	5.5	1.5	3	6	5	6
recommendation level	4	–	4	4	4	4	3.75	2.5	3	4.5	5.5	5.5
guidance level	–	4	5	4	4	4	2.5	2.5	2.5	5.25	4.5	5
Organisational usefulness:												
success ratio	–	4	4	4.5	4	4	3.75	3	3.5	5.25	5	5.5
benefit ratio	4	5	4	5	5	5	4	3.5	4	6.25	5.5	6
enhancement ratio	6	6	6	5	4	4	4	3	4	6	5.5	6

simple judgement heuristics when making a decision and taking mental shortcuts by considering only one aspect of a difficult and complex problem and ignoring the other - possibly also important. Easier and simpler strategies tend to generally win when finding a solution for a complex problem. Cialdini catalogued the judgement heuristics in "six principles of influence" [11]. One of the heuristics that seemed to work in our case was the authority principle. According to this principle, humans tend to comply with people they find to be in a position of authority. A good example is the "power" of uniform showed by Bickman in [5] in which the subjects showed more compliance to people in uniform. Such compliance can also be seen in the case of non-human authorities. A good example is the law or legal precedence shown in [52]. It is plausible to treat a software tool also as a type of non-human authority, especially if, like Jaskier, it is prominently visible in the software developer IDE. What if it was seen by the developer as an authority, even though it showed false positives? The tool (using a prediction model in the background) might have stated that the developer's changes would not cause the build to fail, but in fact they might have caused a problem. A very simple judgement heuristic can be used by the developer: trust the prediction and push the changes to the repository without checking it thoroughly enough.

To investigate further the influence of the authority principle on the experiment results, we conducted a prediction model analysis in terms of false positive predictions. We found that the model used in the experiment can be greatly improved by the proposed algorithm using the mlr3 library or neural networks.

The tool acceptance investigated using the TAM-inspired survey showed that the ItU of Jaskier was higher among the industry experts who never used the tool than among the software developers who took part in the experiment.

8 Conclusions and future work

We have introduced a new practice to predict a failing build on a CI server and provide immediate feedback to the developer in his IDE. This practice uses historical CI server build results, metrics from a source code repository, and continuous software developer feedback. Using the

information on historical build successes and failures and the metrics derived from the software repository at the time the build was performed, we calculated a prediction model that was used during the software developer work. In a just-in-time manner, the metrics for the code that a developer was working on were calculated and sent to the model for prediction. The prediction was then presented to the developer. The idea was that developers, when confronted with a negative prediction, would be more cautious and check their code changes more carefully before introducing them to the repository and causing a new build.

We have developed a set of tools that enable the CBOP practice, which we named Jaskier. Using this tool set, we performed an experiment in which we investigated the influence of CBOP practice on the ratio of failed builds to all other build outcomes. However, as a result of the experiment, we were not able to confirm the hypothesis that "using CBOP reduces FBR". On the contrary, developers, when presented with the build prediction, tended to cause more failed builds than when the prediction was not present. Because of the very small effect size (and because of the trend in CBOPoff baseline), we used more sophisticated methods of effect size measurement. According to the PEM measure, CBOP was "not effective" or at most had a "debatable effectiveness" according to the PAND measure. Additionally, the descriptive statistics suggested low effectiveness of the CBOP practice in the opposite direction. This might indicate that CBOP may in fact lead to an increase in FBR. However, combined with the fact that FBR increased over time when CBOP was turned off (see Fig. 9), we could not make any strong conclusions.

Nevertheless, we have confirmed the acceptance of the practice among the practitioners in a TAM-inspired study. The main concern the developers had was with the quality of the prediction model. We confirmed that the quality of the model can be increased by using more sophisticated algorithms.

As a plausible explanation for the revealed phenomenon, we considered the authority principle. This principle is rarely discussed in the context of software engineering and AI/ML-supported software development. Therefore, our explanation will need rigorous validation to explain the phenomena fully. In this investigation, we should look at the problem from a different perspective and tackle the general cognitive functions of the developer such as learning, thinking, reasoning, remembering, problem solving, decision making, and attention [22].

The data that support the findings of this study are available in "figshare" with the identifier https://doi.org/10.6084/m9.figshare.14222273.

Acknowledgements Calculations in R have been carried out using resources provided by Wroclaw Centre for Networking and Supercomputing (http://wcss.pl), grant No. 578.

Author Contributions M. Kawalerowicz: Conceptualization, Methodology, Software, Validation, Investigation, Data Curation, Writing – original draft, Writing - review & editing, Visualization.
L. Madeyski: Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing - review & editing, Supervision.

References

1. Antoniol G, Ayari K, Di Penta M, Khomh F, Guéhéneuc YG (2008) Is it a bug or an enhancement?: a text-based approach to classify change requests. In: Chechik M., Vigder M, Stewart D (eds) Proceedings of the 2008 conference of the center for advanced studies on collaborative research: meeting of minds, CASCON '08. ACM, New York, pp 23:304–23:318
2. Arora I, Tetarwal V, Saha A (2015) Open issues in software defect prediction. Proc Comput Sci 46:906–912. https://doi.org/10.1016/j.procs.2015.02.161
3. Basili VR, Caldiera G, Rombach HD (1994) The goal question metric approach. In: Marciniak J. J. (ed) Encyclopedia of software engineering. Wiley
4. Bennin K, Ali N, Börstler J, Yu X (2020) Revisiting the impact of concept drift on just-in-time quality assurance. In: Chan W., Nagappan M, Budnik C (eds) 2020 IEEE 20th international conference on software quality, reliability and security (QRS), pp 53–59
5. Bickman L (1974) The social power of a uniform. J Appl Soc Psychol 4:47–61. https://doi.org/10.1111/j.1559-1816.1974.tb02807.x
6. Breiman L (2001) Random forests. Mach Learn 45(1):5–32. https://doi.org/10.1023/A:1010933404324
7. Breiman L, Friedman JH, Olshen RA, Stone CJ (1984) Classification and regression trees. Chapman and Hall, New York
8. Brian R, Terry T, Beth A (2015) Package 'rpart' - Recursive partitioning for classification
9. Bulté I, Onghena P (2009) Randomization tests for multiple-baseline designs: An extension of the SCRT-R package. Behavi Res Methods 41:477–85. https://doi.org/10.3758/BRM.41.2.477
10. Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) SMOTE: synthetic minority over-sampling technique. J Artif Intell Res 16(1):321–357
11. Cialdini R (2009) Influence: the psychology of persuasion. Collins Business Essentials. HarperCollins e-books
12. Criminisi A, Shotton J, Criminisi A, Shotton J (2013) Decision forests for computer vision and medical image analysis. Springer, New York
13. Criminisi A, Shotton J, Konukoglu E (2012) Decision forests: a unified framework for classification, regression, density estimation, manifold learning and semi-supervised learning. NOW Publishers
14. D'Ambros M, Lanza M, Robbes R (2012) Evaluating defect prediction approaches: a benchmark and an extensive comparison. Empir Softw Eng 17(4-5):531–577
15. Davis FD (1989) Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Q 13(3):319–340
16. Davis FD, Bagozzi RP, Warshaw PR (1989) User acceptance of computer technology: a comparison of two theoretical models. Manag Sci 35:982–1003
17. Dugard P, File P, Todman J (2012) Single-case and Small-n experimental designs: a practical guide to randomization tests, 2nd edn. Routledge, Evanston
18. Eken B, Tosun A (2021) Investigating the performance of personalized models for software defect prediction. J Syst Softw 181:111038. https://doi.org/10.1016/j.jss.2021.111038

19. Felix EA, Lee SP (2020) Predicting the number of defects in a new software version. PLoS ONE 15(3):1–30. https://doi.org/10.1371/journal.pone.0229131
20. Ferguson CJ (2009) An effect size primer: A guide for clinicians and researchers. Prof Psychol Res Pract 40(5):532–538
21. Finlay J, Pears R, Connor AM (2014) Data stream mining for predicting software build outcomes using source code metrics. Inf Softw Technol 56(2):183–198. https://doi.org/10.1016/j.infsof.2013.09.001
22. Fisher GG, Chacon M, Chaffee DS (2019) Chapter 2 - theories of cognitive aging and work. In: Baltes BB, Rudolph CW, Zacher H (eds) Work across the lifespan. Academic Press, pp 17–45. https://doi.org/10.1016/B978-0-12-812756-8.00002-5
23. Friedman JH (2000) Greedy function approximation: a gradient boosting machine. Ann Stat 29:1189–1232
24. Hoang T, Kang HJ, Lo D, Lawall J (2020) CC2Vec: distributed representations of code changes. In: Rothermel G, Bae D-H (eds) Proceedings of the ACM/IEEE 42nd international conference on software engineering. Association for Computing Machinery, New York, pp 518–529
25. Hoang T, Khanh Dam H, Kamei Y, Lo D, Ubayashi N (2019) DeepJIT: an end-to-end deep learning framework for just-in-time defect prediction. In: Storey M-A, Adams B, Haiduc S (eds) 2019 IEEE/ACM 16th international conference on mining software repositories (MSR), pp 34–45
26. James G, Witten D, Hastie T, Tibshirani R (2014) An introduction to statistical learning: with applications in R. Springer Publishing Company, Incorporated
27. Jiang L, Jiang S, Gong L, Dong Y, Yu Q (2020) Which process metrics are significantly important to change of defects in evolving projects: an empirical study. IEEE Access 8:93705–93722. https://doi.org/10.1109/ACCESS.2020.2994528
28. Jiang T, Tan L, Kim S (2013) Personalized defect prediction. In: Denney E, Bultan T, Zeller A (eds) 2013 28th IEEE/ACM international conference on automated software engineering (ASE), pp 279–289
29. Kabir MA, Keung J, Turhan B, Bennin K (2021) Inter-release defect prediction with feature selection using temporal chunk-based learning: an empirical study. Appl Soft Comput 113:107870. https://doi.org/10.1016/j.asoc.2021.107870
30. Kamei Y, Shihab E, Adams B, Hassan AE, Mockus A, Sinha A, Ubayashi N (2013) A large-scale empirical study of just-in-time quality assurance. IEEE Trans Softw Eng 39(6):757–773
31. Kawalerowicz M, Madeyski L (2021a) Continuous build outcome prediction: a small-n experiment in settings of a real software project. In: Fujita H, Selamat A, Lin JC-W, Ali M (eds) Advances and trends in artificial intelligence. From theory to practice. Springer International Publishing, Cham, pp 412–425
32. Kawalerowicz M, Madeyski L (2021b) Jaskier: A supporting software tool for continuous build outcome prediction practice. In: Fujita H, Selamat A, Lin JC-W, Ali M (eds) Advances and trends in artificial intelligence. from theory to practice, Springer International Publishing, Cham, 1128 pp 426–438. https://doi.org/10.1007/978-3-030-79463-7_36
33. Kim S, Whitehead EJ Jr, Zhang Y (2008) Classifying software changes: clean or buggy? IEEE Trans Softw Eng 34(2):181–196
34. Kitchenham B, Madeyski L, Budgen D, Keung J, Brereton P, Charters S, Gibbs S, Pohthong A (2017) Robust statistical methods for empirical software engineering. Empir Softw Eng 22(2):579–630. https://doi.org/10.1007/s10664-016-9437-5
35. Kuhn M (2008) Building predictive models in R using the caret package. J Stat Softw 28(5):1–26
36. Lanza M, Mocci A, Ponzanelli L (2016) The tragedy of defect prediction, prince of empirical software engineering research. IEEE Softw 33(6):102–105
37. Liaw A, Wiener M (2015) Package 'randomForest' - Breiman and Cutler's random forests for classification and regression
38. Ma HH (2006) An alternative method for quantitative synthesis of single-subject researches. Behav Modif 30(5):598–617
39. Madeyski L (2010) Test-driven development: an empirical evaluation of agile practice. Springer, New York
40. Madeyski L, Jureczko M (2015) Which process metrics can significantly improve defect prediction models? An empirical study. Softw Qual J 23(3):393–422. https://doi.org/10.1007/s11219-014-9241-7
41. Madeyski L, Kawalerowicz M (2017) Continuous defect prediction: the idea and a related dataset. In: González-Barahona JM, Hindle A, Tan L (eds) 14th international conference on mining software repositories (May 20-21, 2017. Buenos Aires, Argentina), pp 515–518. https://doi.org/10.1109/MSR.2017.46
42. Madhavan JT, Whitehead, EJ Jr (2007) Predicting buggy changes inside an integrated development environment. In: Cheng L-T, Morris C, Orso A, Robillard M (eds) Proceedings of the 2007 OOPSLA workshop on eclipse technology eXchange, eclipse '07. ACM, New York, pp 36–40
43. Marciniak JJ (2002) Encyclopedia of software engineering, 2n. Halsted Press, USA
44. Menzies T, Milton Z, Turhan B, Cukic B, Jiang Y, Bener A (2010) Defect prediction from static code features: current results, limitations, new approaches. Autom Softw Eng 17(4):375–407. https://doi.org/10.1007/s10515-010-0069-5
45. Moser R, Pedrycz W, Succi G (2008) A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction. In: Schäfer W, Dwyer MB, Gruhn V (eds) 2008 ACM/IEEE 30th international conference on software engineering, pp 181–190
46. Onghena P (1992) Randomization tests for extensions and variations of ABAB single-case experimental designs: A rejoinder. Behav Assess 14:153–171
47. Parker RI, Hagan-Burke S, Vannest K (2007) Percentage of all non-overlapping data (PAND): an Alternative to PND. J Spec Educ 40:194–204
48. Saidani I, Ouni A (2021) Toward a smell-aware prediction model for CI build failures. In: Grundy J, Hao D, Poshyvanyk D (eds) 2021 36th IEEE/ACM international conference on automated software engineering workshops (ASEW), pp 18–25
49. Saidani I, Ouni A, Chouchen M, Mkaouer MW (2020a) On the prediction of continuous integration build failures using search-based software engineering. In: Coello CAC (ed) Proceedings of the 2020 genetic and evolutionary computation conference companion, GECCO '20. Association for Computing Machinery, New York, pp 313–314
50. Saidani I, Ouni A, Chouchen M, Mkaouer MW (2020b) Predicting continuous integration build failures using evolutionary search. Inf Softw Technol 128:106392
51. Saidani I, Ouni A, Mkaouer MW (2022) Improving the prediction of continuous integration build failures using deep learning. Autom Softw Eng 29(1):1–61. https://doi.org/10.1007/s10515-021-00319-5
52. Schneider A, Honeyman C (2006) The Negotiator's fieldbook. American Bar Association, Section of Dispute Resolution
53. Sing T, Sander O, Beerenwinkel N, Lengauer T (2005) ROCR: visualizing classifier performance in R. Bioinformatics 21(20):3940. https://doi.org/10.1093/bioinformatics/bti623
54. Turner M, Kitchenham B, Brereton P, Charters S, Budgen D (2010) Does the technology acceptance model predict actual use? A systematic literature review. Inf Softw Technol 52(5):463–479. https://doi.org/10.1016/j.infsof.2009.11.005
55. Venkatesh V, Davis FD (2000) A theoretical extension of the technology acceptance model: Four longitudinal field studies. Manag Sci 46(2):186–204

56. Weyuker E, Ostrand T, Bell R (2008) Do too many cooks spoil the broth? using the number of developers to enhance defect prediction models. Empir Softw Eng 13:539–559. https://doi.org/10.1007/s10664-008-9082-8
57. Wohlin C, Runeson P, Höst M, Ohlsson M, Regnell B, Wesslén A (2012) Experimentation in Software Engineering. Computer Science. Springer
58. Wright MN, Zieglerm A (2017) ranger: A fast implementation of random forests for high dimensional data in C++ and R. J Stat Softw 77(1):1–17. https://doi.org/10.18637/jss.v077.i01
59. Yan M, Xia X, Fan Y, Hassan AE, Lo D, Li S (2022) Just-in-time defect identification and localization: a two-phase framework. IEEE Trans Softw Eng 48(1):82–101. https://doi.org/10.1109/TSE.2020.2978819
60. Yang X, Lo D, Xia X, Sun J (2017) TLEL: A two-layer ensemble learning approach for just-in-time defect prediction. Inf Softw Technol 87:206–220
61. Zeng Z, Zhang Y, Zhang H, Zhang L (2021) Deep just-in-time defect prediction: How far are we? In: Cadar C, Zhang X (eds) Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis, ISSTA 2021. Association for Computing Machinery, New York, pp 427–438

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Marcin Kawalerowicz is the owner of CODEFUSION, a software engineering firm located in Opole, Poland. With more than 20 years of experience in developing business-oriented software for clients across Europe and the United States, together with his team, develops high-end computer software. He also writes articles for scientific and popular computer magazines and is a frequent speaker at conferences. He is a co-author of a book "Continuous Integration in .NET".

Marcin Kawalerowicz holds a Master's degree in Computer Science from the Technical University of Opole in Poland. He is currently an aspiring researcher pursuing a PhD in Software Engineering, applying his expertise towards advancing the field.



Lech Madeyski is an Associate Professor and Deputy Head of the Department of Applied Informatics at Wroclaw University of Science and Technology, Poland. He has been a Visiting Researcher or Professor at Keele University, Brunel University London, Lancaster University, and Blekinge Institute of Technology. His research focuses on empirical software engineering, data science in software engineering, reproducible research, robust statistical methods. He is a co-founder of *e-Informatica Software Engineering Journal* and International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE). He has published in prestigious journals including series of articles, e.g., *IEEE Transactions on Software Engineering*, *Empirical Software Engineering*, *Information and Software Technology*, and *Statistics in Medicine*. He is an author of a book "Test-Driven Development: An Empirical Evaluation of Agile Practice" including meta-analysis of experiments.